WAS Experiment 05

Aim: Attacking Session and Authentication in Web Applications

Learning Objective: To demonstrate common weaknesses in web application authentication and session management in a controlled laboratory environment and understand how improper authentication and session handling can lead to unauthorized access.

Learning Objectives:
After completing this experiment, students will be able to:

- Understand authentication and session management.
- Explain the difference between authentication and authorization.
- Understand how web applications maintain user sessions.
- Analyze HTTP authentication requests and responses.
- Examine session cookies.
- Demonstrate common authentication weaknesses in a controlled environment.
- Understand session-related security risks.
- Identify basic security measures for protecting authentication and sessions.
- Apply secure authentication and session-management practices.

Tools: Kali Linux, Terminal, Burp Suite, Web Browser-Firefox and Local or authorized vulnerable web application.

Theory:

Introduction

Authentication is the process of verifying the identity of a user. Web applications commonly use usernames and passwords to authenticate users.
After successful authentication, the application generally creates a **session** so that the user does not need to provide their credentials with every request.
A session is commonly maintained using a session identifier stored in a browser cookie.

Authentication

Authentication verifies who the user is.
Examples include:

- Username and password
- Multi-factor authentication
- Security tokens
- Biometric authentication

A secure authentication system should properly validate credentials and protect them from unauthorized disclosure.

Authorization

Authorization determines what an authenticated user is allowed to access.
For example:
Authentication:
"Who are you?"

Authorization:
"What are you allowed to access?"
Authentication and authorization are different security mechanisms and both must be implemented correctly.

Session Management
After successful authentication, a web application may create a session identifier.
For example:
Set-Cookie: sessionid=ABC123
If session identifiers are predictable, exposed, improperly protected, or not invalidated after logout, attackers may be able to abuse sessions.

Common Authentication and Session Weaknesses
1. Weak Passwords
Applications that permit easily guessable passwords are more vulnerable to credential attacks.
2. Weak Authentication Controls
Improper authentication logic may allow unauthorized users to access protected functionality.
3. Session Fixation
Session fixation occurs when an attacker can cause a victim to use a session identifier known to the attacker.
4. Session Hijacking
Session hijacking involves obtaining or abusing another user's valid session identifier.
5. Improper Session Invalidation
If a session remains valid after logout or for an unnecessarily long period, it may increase security risk.
6. Insecure Cookies
Session cookies should use appropriate security attributes such as:

- Secure
- HttpOnly
- SameSite

Procedure:

Step 1: Start Kali Linux

Start Kali Linux and open Burp Suite.

burpsuite

Select Temporary Project and continue using the default configuration.

Expected Output:

Burp Suite opens successfully.

Step 2: Configure Firefox

Configure Firefox to use Burp Suite as the local proxy:

Proxy Address: 127.0.0.1

Proxy Port: 8080

Expected Output:

Firefox traffic is routed through Burp Suite.

Step 3: Open the Authorized Web Application

Open a local or intentionally vulnerable web application.

Example:

http://localhost/

Only use an application that is specifically authorized for security testing.

Expected Output:

The login page of the web application is displayed.

Step 4: Perform Normal Login

Enter valid test credentials provided by the laboratory application.

For example:

Username: testuser

Password: testpassword

Submit the login form.

Expected Output:

The application authenticates the user and provides access to the authenticated area.

Step 5: Capture the Login Request

In Burp Suite, open:

Proxy → Intercept

Capture the login request generated by the browser.

A request may contain:

POST /login HTTP/1.1

Host: localhost

Content-Type: application/x-www-form-urlencoded

username=testuser&password=testpassword

Expected Output:

The authentication request is displayed in Burp Suite.

Step 6: Analyze the Authentication Request

Examine:

- HTTP method
- URL
- Username parameter
- Password parameter
- HTTP headers
- Cookies
- Other request parameters

Expected Output:

The components of the authentication request can be identified.

Step 7: Analyze the Session Cookie

After successful authentication, inspect the response.

Look for a cookie such as:

Set-Cookie: sessionid=ABC123

Check whether appropriate cookie security attributes are present.

For example:

Secure

HttpOnly

SameSite

Expected Output:

The application's session-management mechanism and cookie attributes can be analyzed.

Step 8: Test Logout and Session Invalidation

Log out of the application.

Then attempt to access an authenticated page using the same browser session.

Expected Output:

A properly configured application should prevent access after logout and require authentication again.

Step 9: Test Session Timeout

Remain inactive for the application's configured session period, if practical in the laboratory environment.

After the session expires, attempt to access a protected resource.

Expected Output:

The application should require the user to authenticate again after session expiration.

Step 10: Test Authentication Controls

Using the authorized test account, attempt incorrect credentials and observe the application's behavior.

Observe whether the application:

- Rejects invalid credentials.
- Provides appropriate error messages.
- Limits repeated failed attempts.
- Implements appropriate account protections.

Expected Output:

Invalid authentication attempts are rejected and appropriate security controls are observed.

Step 11: Analyze Requests Using Burp Repeater

Send an authorized test request to Burp Repeater.

Modify a harmless parameter and resend the request.

Observe whether the application properly validates the request and maintains the expected authorization boundaries.

Expected Output:

The application response to the modified request can be analyzed.

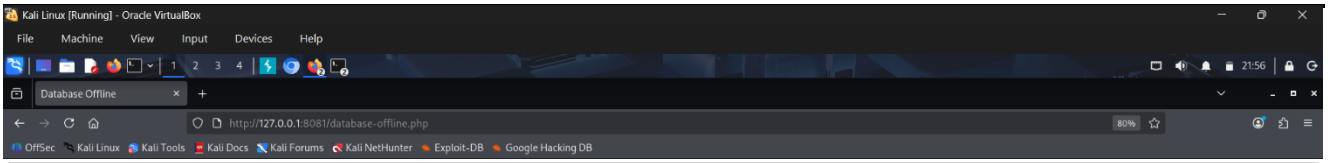
Outputs:

```
┌──(kali㉿kali)-[~]
└─$ sudo docker ps -a
[sudo] password for kali:
CONTAINER ID   IMAGE                  COMMAND                  CREATED      STATUS      PORTS                                                NAMES
89f7d03776fd   bkimminich/juice-shop  "/nodejs/bin/node /j…"   2 days ago   Up 2 days   0.0.0.0:3000→3000/tcp, [::]:3000→3000/tcp            juice-shop

┌──(kali㉿kali)-[~]
└─$ sudo docker pull citizenstig/nowasp
Using default tag: latest
latest: Pulling from citizenstig/nowasp
8387d9ff0016: Pull complete
3b52deaaf0ed: Pull complete
4bd501fad6de: Pull complete
a3ed95caeb02: Pull complete
790f0e8363b9: Pull complete
11f87572ad81: Pull complete
341e06373981: Pull complete
709079cecfb8: Pull complete
55bf9bbb788a: Pull complete
b41f3cfd3d47: Pull complete
70789ae370c5: Pull complete
43f2fd9a6779: Pull complete
6a0b3a1558bd: Pull complete
934438c9af31: Pull complete
1cfba20318ab: Pull complete
de7f3e54c21c: Pull complete
596da16c3b16: Pull complete
e94007c4319f: Pull complete
3c013e645156: Pull complete
e75bb486a4a4: Pull complete
a606d06e79b9: Pull complete
Digest: sha256:8bf6f283163bbc269ccfff513e4a06776325cb4957f683df9edd4ab98c6966de
Status: Downloaded newer image for citizenstig/nowasp:latest
docker.io/citizenstig/nowasp:latest
```

```
┌──(kali㉿kali)-[~]
└─$ sudo docker run -d --name mutillidae -p 8081:80 citizenstig/nowasp
8779882b73a5bfacd259ab8d0414e86bd21f45e2b44aa9d2ae4d7d400e147cb8

┌──(kali㉿kali)-[~]
└─$ sudo docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED              STATUS              PORTS                                                NAMES
8779882b73a5   citizenstig/nowasp     "/run.sh"                About a minute ago   Up About a minute   3306/tcp, 0.0.0.0:8081→80/tcp, [::]:8081→80/tcp      mutillidae
89f7d03776fd   bkimminich/juice-shop  "/nodejs/bin/node /j…"   2 days ago           Up 2 days           0.0.0.0:3000→3000/tcp, [::]:3000→3000/tcp            juice-shop
```



The database server appears to be offline.

The database server at 127.0.0.1 appears to be offline.

1. Be sure the username and password to MySQL is the same as configured in includes/database-config.php
2. Be aware that MySQL disables password authentication for root user upon installation or update in some systesms. This may happen even for a minor update. Please check the username and password to MySQL is the same as configured in includes/database-config.php
3. Try to **setup/reset the DB** to see if that helps
4. **A video is available** to help reset MySQL root password
5. The commands vary by system and version, but may be something similar to the following
   - mysql -u root
   - use mysql;
   - update user set authentication_string=PASSWORD('') where user='root';
   - update user set plugin='mysql_native_password' where user='root';
   - flush privileges;
   - quit;
6. Check the error message below for more hints
7. If you think this message is a false-positive, you can opt-out of these warnings below

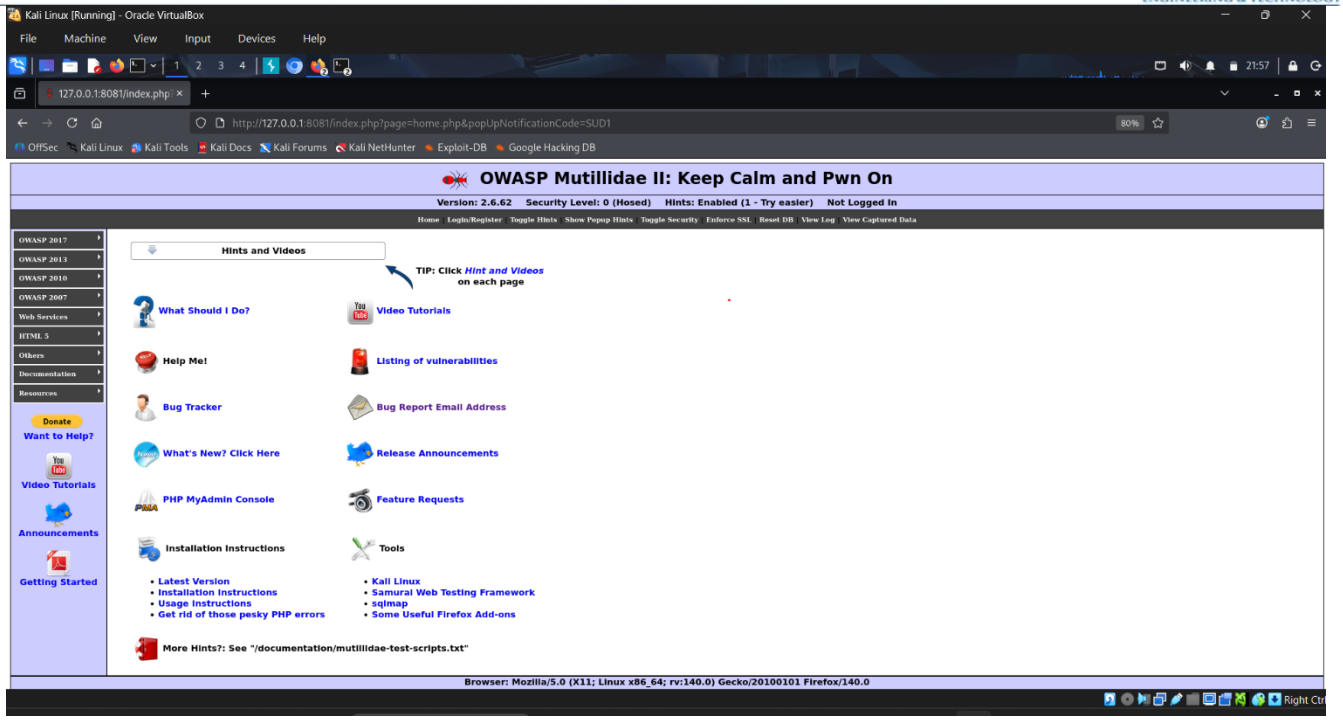
Error Message

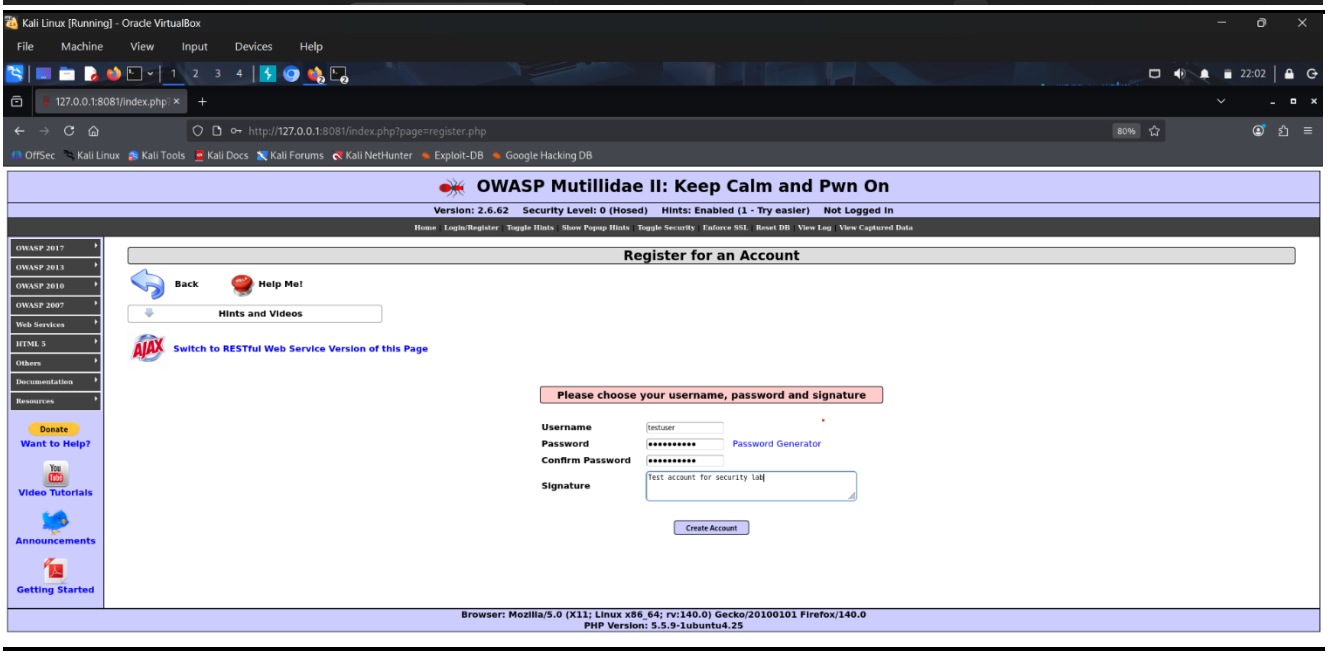
Error: Failed to connect to MySQL database. Unable to select default database mutillidae. It appears that the database to which Mutillidae is configured to connect has not been created. Try to setup/reset the DB to see if that helps. Next, check that the database service is running and that the database username, password, database name, and database location are configured correctly. Note: File /mutillidae/classes/MySQLHandler.php contains the database configuration. Connection error:

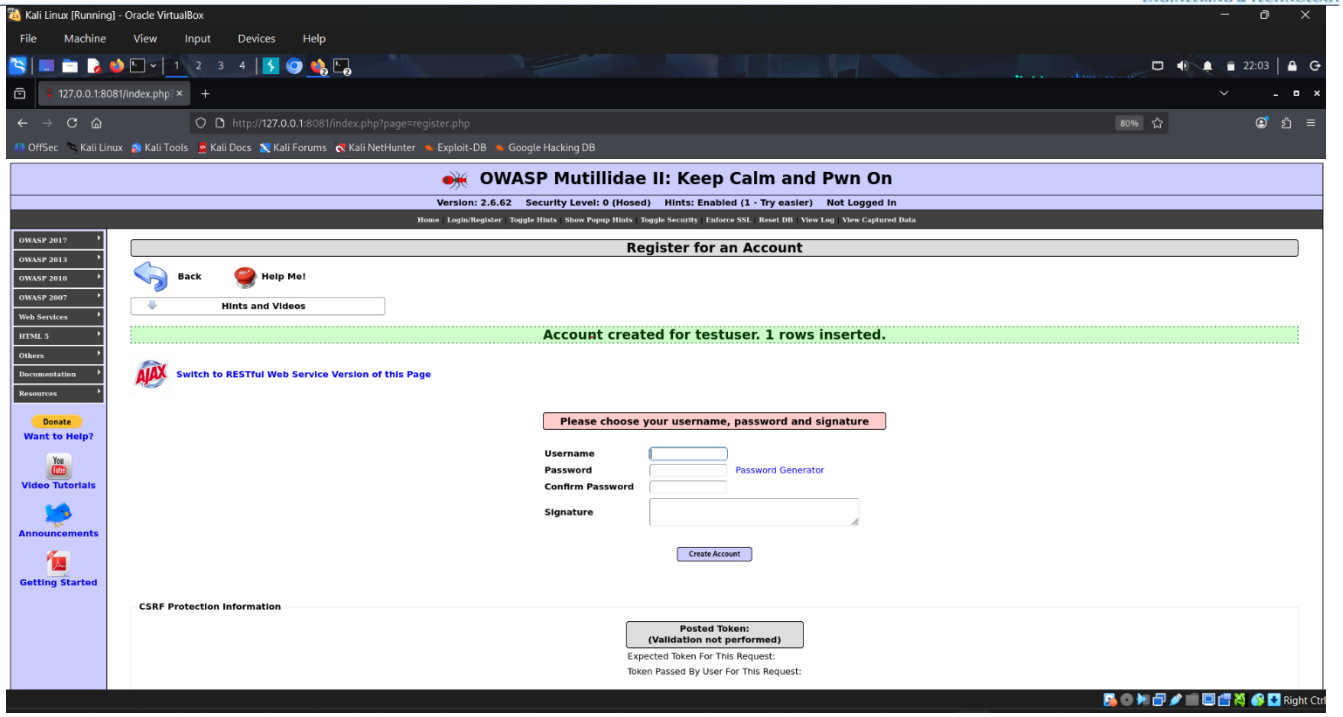
Opt out of database warnings

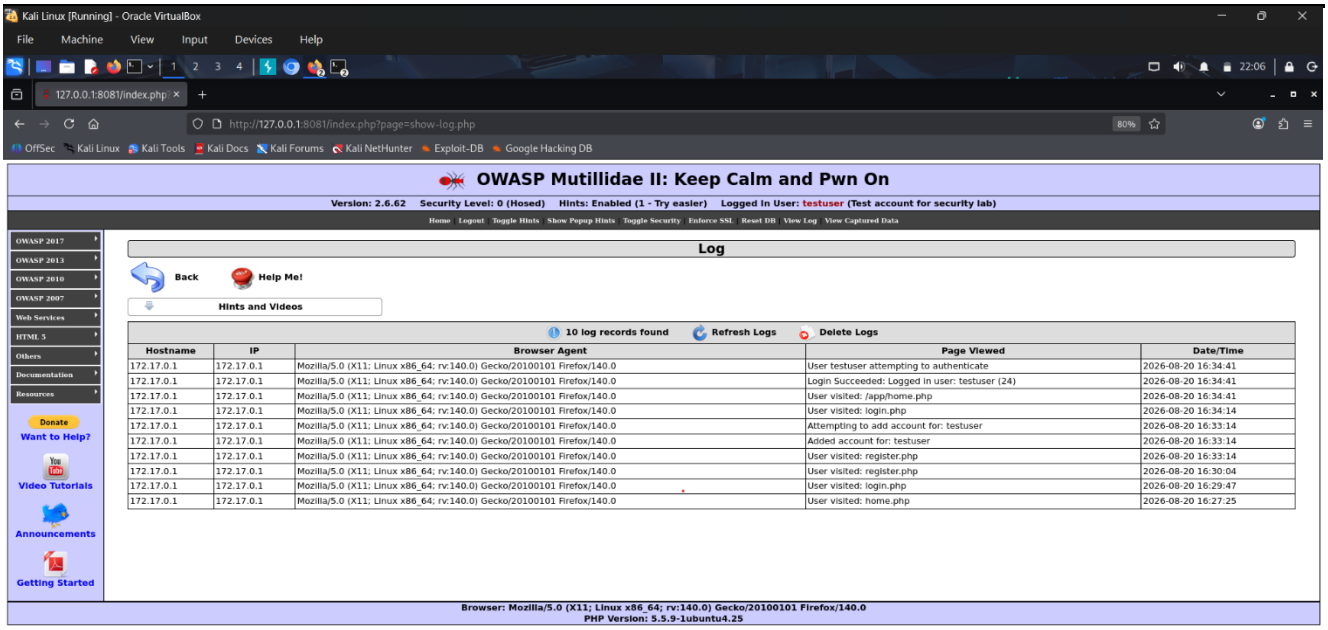
You can opt out of database connection warnings for the remainder of this session

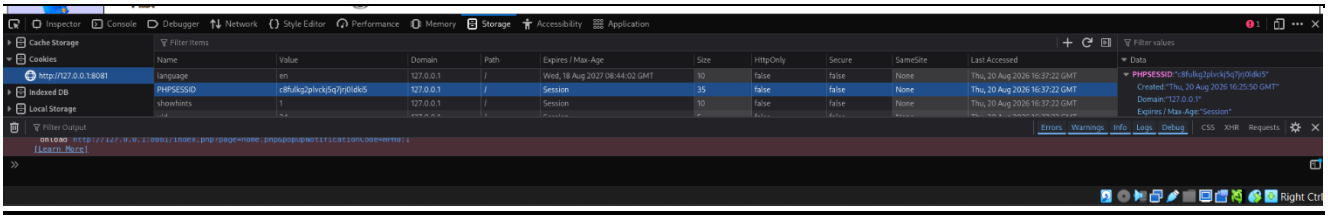
Opt Out


Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
http://127.0.0.1:8081/index.php?page=home.php&popUpNotificationCode=SUD1
OWASP Mutillidae II: Keep Calm and Pwn On
Version: 2.6.62 Security Level: 0 (Hosed) Hints: Enabled (1 - Try easier) Not Logged In
Home Login/Register Toggle Hints Show Popup Hints Toggle Security Enforce SSL Reset DB View Log View Captured Data
Hints and Videos
TIP: Click Hint and Videos on each page
What Should I Do?
Video Tutorials
Help Me!
Listing of vulnerabilities
Bug Tracker
Bug Report Email Address
What's New? Click Here
Release Announcements
PHP MyAdmin Console
Feature Requests
Installation Instructions
Tools
Latest Version
Installation Instructions
Usage Instructions
Get rid of those pesky PHP errors
Kali Linux
Samurai Web Testing Framework
sqlmap
Some Useful Firefox Add-ons
More Hints?: See "/documentation/mutillidae-test-scripts.txt"
Browser: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0


Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
http://127.0.0.1:8081/index.php?page=register.php
OWASP Mutillidae II: Keep Calm and Pwn On
Version: 2.6.62 Security Level: 0 (Hosed) Hints: Enabled (1 - Try easier) Not Logged In
Register for an Account
Back
Help Me!
Hints and Videos
Switch to RESTful Web Service Version of this Page
Please choose your username, password and signature
Username
testuser
Password
Password Generator
Confirm Password
Signature
Test account for security lab
Create Account
Browser: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
PHP Version: 5.5.9-1ubuntu4.25







Observation:

The following observations are made:

1. Authentication requests contain user credentials or authentication-related parameters.

2. Successful authentication results in the creation or assignment of a session.
3. Session cookies are used to maintain authenticated state.
4. Burp Suite can be used to inspect authentication requests and responses.
5. Cookie security attributes can be examined.
6. Logout should invalidate the authenticated session.
7. Expired sessions should not continue to provide access to protected resources.
8. Proper authentication and session-management controls reduce the risk of unauthorized access.

Result and Discussion:

The experiment successfully demonstrated the analysis of authentication and session management in a web application.

Burp Suite was used to intercept and inspect authentication requests, responses, and session cookies. The experiment demonstrated how authenticated sessions are established and maintained through session identifiers.

The experiment also showed the importance of proper authentication controls, session expiration, logout handling, and secure cookie configuration. Weaknesses in these mechanisms can expose web applications to authentication and session-related attacks.

Conclusion:

This experiment demonstrated how authentication and session management operate in web applications and how Burp Suite can be used to analyze their implementation.

The experiment emphasized the importance of secure authentication mechanisms, protected session cookies, proper session invalidation, session expiration, and appropriate access controls for preventing unauthorized access.

Viva Questions:

1. What is authentication?
2. What is session management in a web application?
3. What is the difference between authentication and authorization?

Name: Aarna Singh

BE CSE

Roll No. 02

For Faculty Use:

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				